

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 03 · LECTURE NOTES

Balls and Bins

The model behind hashing, load balancing, and caching.

MODULE OUTCOMES

- Derive the maximum load for n balls in n bins.
- Explain the birthday and coupon-collector results and their uses.
- Explain the power of two choices and why it is so effective.
- Apply the model to hashing and load balancing.
- Explain where the model's assumptions fail in practice.

1 The basic results

Question	Answer	What it is used for
Maximum load, n balls in n bins	$\Theta(\log n / \log \log n)$	Worst-case hash chain length , and the caveat on 'O(1) lookup'.
Number of empty bins	About n/e , which is roughly 37%	Hash table occupancy — and why a table at load factor 1 wastes more than a third of its slots.
When the first collision occurs	After about $\sqrt{n}$ balls	Birthday attacks and hash output sizing (CSCE 711).
Balls needed to hit every bin	About $n \ln n$	The coupon collector problem; cache warming periods; test coverage by random sampling.
Maximum load with $n \log n$ balls	Concentrated within $O(\sqrt{(\log n)})$ of the mean	Why over-provisioning works — see §4.

The first and third are the ones to remember: maximum load is $\log n / \log \log n$, and collisions start at $\sqrt{n}$ — and both are counterintuitive in the useful direction. The maximum load is larger than people expect (so hash tables have longer worst-case chains than the average suggests) and the collision point is much earlier than people expect (so hash outputs need twice the bits you would naively want).

```

P(a specific bin receives  $\geq k$  balls)
   $\leq C(n,k) * (1/n)^k$       # choose which k balls land
   $\leq 1/k!$                   # since  $C(n,k) \leq n^k / k!$ 
   $\leq (e/k)^k$               # Stirling

```

UNION BOUND over the n bins:

```

P(any bin receives  $\geq k$ )  $\leq n * (e/k)^k$ 

```

We want this below $1/n$, so we need $(e/k)^k \leq 1/n^2$, i.e.
 $k * \log(k/e) \geq 2 \log n$.

Setting $k = c \log n / \log \log n$ satisfies that for a suitable constant c . So WITH HIGH PROBABILITY the maximum load is $O(\log n / \log \log n)$.

AND A MATCHING LOWER BOUND holds, so the answer is tight rather than merely an upper bound.

NOTE THE STRUCTURE: a per-bin bound, then a union bound over bins, with the per-bin probability deliberately driven to $1/n^2$ so that the union over n bins leaves $1/n$. That is EXACTLY Module 02 section 3's pattern -- and almost every result in Modules 03 through 05 has this shape.

2 The power of two choices

TWO RANDOM CHOICES INSTEAD OF ONE CHANGES $\log n / \log \log n$ TO $\log \log n$

Pick *two* bins uniformly at random and put the ball in whichever is currently less loaded. The maximum load drops to $\log \log n / \log 2 + O(1)$.

At $n =$ one million, that is about 20 against about 4 — an exponential improvement obtained from one extra random choice and one comparison per insertion. It is one of the best cost-benefit ratios in algorithms.

And $d = 3$ choices gives $\log \log n / \log 3$, which is a constant-factor further improvement — so almost all the benefit is in the second choice, and there is little reason to go beyond two or three.

Which is why it is used everywhere: load balancers, distributed hash tables, cuckoo hashing, work-stealing schedulers, and connection pools all use two-choice selection (Module 11 §3, CSCE 678), and this theorem is the reason rather than folklore.

WHY TWO CHOICES WORKS, INTUITIVELY

With one choice, a bin's load grows whenever it happens to be picked — so the load distribution's tail is governed by a straightforward Poisson-like process, with the tail decaying only as fast as factorials.

With two choices, a bin grows only if it is picked *and* is the lighter of the two — so a heavily loaded bin is very unlikely to grow further, because its partner will usually be lighter.

The effect compounds up the load distribution: the fraction of bins with load at least $k+1$ is roughly the *square* of the fraction with load at least k , because the ball must hit two bins both already at load k . Which gives doubly exponential decay in k .

And that squaring is exactly where the second logarithm comes from — the number of squarings needed to drive a fraction from $1/2$ down below $1/n$ is $\log \log n$. Which is a satisfying derivation to carry rather than a formula to recall, and it also tells you immediately that d choices gives $\log \log n / \log d$.

3 Applications

- **Hash table chain lengths.** The maximum chain is $\Theta(\log n / \log \log n)$ with random hashing, which is why a hash lookup is described as $O(1)$ with a caveat — the average is constant and the worst case over n lookups is not.
- **Birthday attacks.** Collisions beginning at $\sqrt{n}$ means a 128-bit hash provides only 64 bits of collision resistance (CSCE 711) — which is the single most consequential application of this model, and is why hash output sizes are doubled relative to the security level wanted.
- **Cache warming and the coupon collector.** About $n \ln n$ random accesses are needed to touch n distinct items, which sizes a warm-up period or a random-sampling test campaign — and the $\ln n$ factor is frequently forgotten.
- **Load balancing.** Two-choice selection, essentially universally, in software load balancers and in distributed task queues (Module 11 §3).
- **And sharding.** The maximum-load result is why uniform hashing still produces hot shards, and why consistent hashing implementations use many virtual nodes per physical server — which is the model applied deliberately: many small bins per server concentrates the per-server total far better than one bin each, by the $n \log n$ row of §1's table.

4 Where the model fails

REAL WORKLOADS ARE NOT UNIFORM, AND THE MODEL'S PREDICTIONS CHANGE

The analysis assumes balls are thrown uniformly and independently. Real keys are skewed — Zipf distributed, in most systems with human-generated traffic — so a few bins receive a constant fraction of the traffic regardless of how good the hash function is.

Which means the maximum-load result simply does not apply to hot keys, and the engineering response is correspondingly different: replication, caching in front, or splitting the key — not a better hash function, which cannot help.

And the balls are frequently not independent either: sequential identifiers, timestamps, and structured keys all correlate, and a weak hash function will preserve the correlation rather than destroy it — which is a real cause of unexpected clustering.

So the practical procedure is: measure the actual load distribution rather than assuming the model, and use the model to tell you what *uniform* would have looked like — which is the right use of it: a baseline for comparison rather than a prediction. 'Uniform would give 20 and we observe 400' is a diagnosis, and it is a far more useful output than either number alone.

- Use it as a baseline, not a prediction. 'Uniform hashing would give a maximum load of 20; we observe 400' is a diagnosis that points at key skew or a bad hash.

- **Over-provision by a logarithmic factor if you need concentration. $n \log n$ balls into n bins concentrates tightly** (§1's last row) — **which is why larger hash tables behave better than the load-factor ratio suggests**, and is an argument for more shards rather than bigger ones.
- **Use two choices wherever the second choice is affordable — which it almost always is**, since it costs one extra lookup and one comparison for an exponential improvement in the tail.
- **And use many small bins rather than few large ones** when you control the mapping — the virtual-node trick, which is the same over-provisioning argument applied to the bin count instead of the ball count.
- **Then measure the tail. The maximum load, not the mean, is what causes the incident** (CSCE 678 Module 06) — **and 'measure the tail' is the recurring instruction of this program. What this module adds is the number the tail *should* be**, which is what makes the measurement interpretable rather than merely a number.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mitzenmacher & Upfal — chapters 5 and 14	Balls and bins, the Poisson approximation, and the power of two choices — the reference for this module.
Azar, Broder, Karlin & Upfal — Balanced Allocations (free)	§2's result, in the original — the paper that introduced the power of two choices.
Mitzenmacher — The Power of Two Choices: A Survey (free)	§2 and §3, surveyed — including the applications and the variants.
Karger, Lehman et al. — Consistent Hashing (free)	§3's last row , and the virtual-node technique in its original setting.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Simulate n balls into n bins** for n up to 10^6 and plot the maximum load against $\log n / \log \log n$.
- 2 **Measure the fraction of empty bins** and compare against $1/e$.
- 3 **Find the first collision empirically** for several n and confirm the $\sqrt{n}$ scaling.
- 4 **Measure the coupon-collector time** and compare against $n \ln n$.
- 5 **Implement two-choice allocation** and plot its maximum load against one-choice on the same axes.
- 6 **Confirm the $\log \log n$ scaling** and the diminishing return at $d = 3$.

- 7 **Verify the squaring intuition** by measuring the fraction of bins at each load level.
- 8 **Throw Zipf-distributed balls** and report how the maximum load changes.
- 9 **Use a weak hash on sequential keys** and observe the clustering.
- 10 **Implement virtual nodes** and measure the per-server load concentration against one bin per server.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the five numbers for n balls in n bins.
- 2 Which two matter most, and why are both counterintuitive?
- 3 Derive the maximum load bound and name the reusable shape.
- 4 State the power of two choices and the improvement at $n = 10^6$.
- 5 Explain the squaring intuition and where $\log \log n$ comes from.
- 6 Why does a 128-bit hash give 64 bits of collision resistance?
- 7 Name five applications of the model.
- 8 Where does the model fail, and what is the correct response to hot keys?
- 9 Give five practical rules for using the model.